

JAVA 语言实验指导书

2019-2020 学年 第二学期

计算机学院

王宇

yuwang@gzhu.edu.cn

Java 语言实验指导书

计算机科学与网络工程学院

樊志平编写

说明

一、“JAVA 语言实验指导书”供所有选修了本课程的同学使用，本课程的实验有三个部分（注：需要上交三份实验报告），每个部分下设若干个小实验。三个部分包括：

- Java 语言面向对象编程技术基础（6 学时）
- Java 图形图像和多线程应用（6 学时）
- Java 综合应用实验（4 学时）

二、实验指导书中每个小实验包含多个示例程序供大家在实验课上学习和实践。其中，普通黑色字体的实验题目，不需要写入实验报告；红色字体实验是需要大家独立完成并把源代码和实验运行截图写入实验报告。实验报告格式由老师指定。

三、实验报告提交说明：一共需要提交三份报告，每份报告转换成一个 PDF 文件上传。

目录

实验 1	熟悉 Java 程序的开发步骤.....	4
实验 2	熟悉 Java 程序的基本结构.....	6
实验 3	简单型变量输入输出.....	9
实验 4	分支和循环语句.....	13
实验 5	类与对象.....	15
实验 6	类变量与实例变量.....	18
实验 7	子类与继承.....	20
实验 8	文件读写.....	24
实验 9	面向接口编程.....	26
实验 10	实验报告（一）题目.....	28
实验 11	Java 图形界面的组件与布局.....	29
实验 12	事件处理.....	32
实验 13	算术测试程序.....	35
实验 14	Java 异常类.....	39
实验 15	StringTokenizer 类.....	41
实验 16	输入输出流.....	43
实验 17	多线程处理.....	45
实验 18	实验报告（二）题目.....	49
实验 19	网络应用.....	51
实验 20	实验报告（三）题目.....	55

第一部分

JAVA 语言面向对象编程技术基础

实验1 熟悉 JAVA 程序的开发步骤

1.1 实验目的

本实验的目的是掌握开发 Java 应用程序的三个步骤：编写源文件、编译源文件和运行应用程序。

1.2 实验要求

编写一个简单的 Java 应用程序。该程序在命令行窗口输出两行文字："你好，很高兴学习 Java 语言"和"We are students"。

1.3 程序模版

请按模版要求，将【代码】字段替换为 Java 程序代码。

Hello.java

```
public class Hello{
    public static void main (String args[ ]){
        【代码 1】        //命令行窗口输出"你好，很高兴学习 Java语言"
        Student zhang = new Student();
        zhang.speak();
    }
}
class Student{
    void speak(){
        【代码 2】        //命令行窗口输"We are students"
    }
}
```

1.4 实验指导

- (1) 打开一个文本编辑器

如果是 Windows 操作系统，打开“记事本”编辑器。可以通过“开始”-“程序”-“附件”-“记事本”来打开文本编辑器。如果是其他操作系统，请在指导老师的帮助下打开一个纯文本编辑器。按照“程序模板”的要求编辑输入源程序，在命令行输出“你好，很高兴学习 Java 语言”的代码是：System.out.println("你好，很高兴学习 Java 语言")或 System.out.print("你好，很高兴学习 Java 语言")。二者的区别是，前者在输出字符序列“你好，很高兴学习 Java 语言”后，将输出光标移动到下一行，后者不移动输出光标到下一行。

- (2) 保存源文件并命名为 Hello.java。

要求将源文件保存到 C:盘的某个文件夹中。例如 C:\1000。

(3) 编译源文件

打开命令行窗口，对于 Windows 操作系统，打开 MS-DOS 窗口。可以通过单击“开始” - “程序” - “附件” - “MS-DOS” 打开命令行窗口，也可以单击“开始” 按钮，选择“运行” 命令。在弹出的对话框的输入命令栏中输入 `cmd` 打开命令行窗口。如果当前 MS-DOS 窗口显示的逻辑符是“D:\”，请输入“C:\” 回车确认，使得当前 MS-DOS 窗口的状态是“C:\”。如果目前 MS-DOS 窗口的状态是 C 盘符的某个子目录，请输入“`cd\`”，使得当前 MS-DOS 窗口的状态是“`cd\`”，MS-DOS 窗口的状态是“C:\” 时，输入进入文件夹目录的命令，例如“`CD 1000`”。然后执行下列编译命令。

```
C:\1000> javac Hello.java
```

初学者在这一步可能会遇到下列错误提示。

- Command not Found 出现该错误的原因是没有设置好系统变量 `path`。
- File not Found 出现该错误的原因是没有将源文件保存在当前目录中，例如 C:\1000，或源文件的名字不符合有关规定，例如，错误地将源文件命名为“`hello.java`” 或“`Hello.java.txt`”，要特别注意：Java 语言的标识符号是区分大小写的。
- 出现一些语法错误提示，例如，在汉语输入状态下输入了程序中需要的语句分号等，Java 源程序中语句所涉及的小括号及标点符号都是英文状态下输入的，比如“你好，很学习 Java 语言” 中的引号必须是英文状态下的引号，而字符串里面的符号不受汉字或英文的限制。

(4) 运行程序

```
C:\1000> java Hello
```

这一步可能会遇到错误提示：Exception in thread "main" java.lang.NoClassFoundError。出现该错误的原因是没有设置好系统变量 `classpath` 或运行的不是主类的名字或程序没有主类。JDK 安装目录的 `jre` 文件夹中包含着 Java 应用程序运行时所需要的 Java 类库，这些类库被包含在 `jre\lib` 中的压缩文件 `rt.jar` 中。安装 JDK 一般不需要设置环境变量 `classpath`，如果主类正确，但程序仍然遇到错误提示，例如：Exception in thread "main" java.lang.NoClassFoundError，那么可以重新编辑系统环境变量 `classpath` 的值。对于 Windows 7/XP，右击“计算机” 选择“我的电脑”，在弹出的菜单中选择“属性” - “系统特性” - “高级系统设置” - “高级” 选项，然后单击“环境变量” 按钮，添加系统环境变量，如果曾经设置过环境变量 `classpath`，可单击该行编辑操作，将需要的值加入即可。

注：环境变量 `classpath` 可以加载应用程序当前目录及其子目录中的类。`rt.jar` 包含了 Java 运行环境提供的类库中的类。

1.5 扩展练习

- 编译器怎样提示丢失大括号的错误？
- 编译器怎样提示语句丢失分号的错误？
- 编译器怎样提示将 `System` 写成 `system` 这一错误？
- 编译器怎样提示将 `String` 写成 `string` 这一错误？

实验2 熟悉 JAVA 程序的基本结构

2.1 实验目的

熟悉 Java 应用程序的基本结构，并能联合编译应用程序所需要的类。

2.2 实验要求

编写 3 个源文件：MainClass.java、A.java 和 B.java，每个源文件只有一个类。MainClass.java 含有应用程序的主类（含有 main 方法），并使用了 A 和 B。将 3 个源文件保存到同一目录中，例如 C:\1000，然后编译 MainClass.iava。

2.3 程序模板

请按模板要求，将【代码】替换为 Java 程序代码。

MainClass.java

```
public class MainClass {  
    public static void main (String args[ ]) {  
        【代码 1】        //命令行窗口输出"你好，只需编译我"  
        A a = new A();  
        a.fA();  
        B b = new B();  
        b.fB();  
    }  
}
```

A.java

```
public class A {  
    void fA() {  
        【代码 2】        //命令行窗口输出"I am A"  
    }  
}
```

B.java

```
public class B {  
    void fB() {  
        【代码 3】        //命令行窗口输出"I am B"  
    }  
}
```

2.4 实验指导

编译 MainClass.java 的过程中，Java 系统会自动地先编译 A.java、B.java。编译通过后，C:\1000 中将会有 MainClass.class、A.class 和 B.class 3 个字节码文件。当运行上述 Java 应用程序时，虚拟机将 MainClass.class 和 A.class、B.class 加载到了内存。当虚拟机将 MainClass.class 加载到内存时，就为主类中的 main 方法分配了入口地址，以便 Java 解释器调用 main 方法开始运行程序。如果编写程序时错误地将主类中的 main 方法写成：public void main(String args[]），那么，程序可以编译通过，但无法运行。

2.5 扩展练习

2.5.1 思考

将 MainClass.class 编译通过以后，不断地修改 A.java 源文件中的代码，比如，在命令行窗口输出 "Nice to meet you!" 或 "Can you help me?"。要求每次修改 A.java 源文件以后，单独编译 A.java，然后直接运行应用程序 MainClass。

2.5.2 对象赋值实验

按照以下要求编写 Java 应用程序：定义一个类 Point，用来描述平面直角坐标系中点的坐标，该类应该能描述点的横、纵坐标信息及一些相关操作，包括获取点的横、纵坐标，修改点的坐标，显示点的当前位置等；定义一个主类 PointTest，创建 Point 类的对象并对其进行有关的操作。理解 Java 对象的理解“值传递”的过程。

PointTest.java

```
class Point{
    double x,y;
    public void setXY(double a, double b){
        x=a;
        y=b;
    }
    public double getX(){
        return x;
    }
    public double getY(){
        return y;
    }
    public void disp(){
        System.out.println("点的当前坐标为: (" +x+", "+y+")");
    }
}
public class PointTest{
    public static void main(String[] args){
        Point p1=new Point();
        p1.disp();
        p1.setXY(3.2,5.6);
        p1.disp();
    }
}
```

运行结果：

- 点的当前坐标为：(0.0, 0.0)
- 点的当前坐标为：(3.2, 5.6)

2.5.3 JAVADOC 文档化工具的使用

Java 2 SDK 1.4.1 中提供了一个文档自动生成工具，可以简化程序员编写文档的工作。可以使用 javadoc.exe 命令启动 Java 文档化工具，自动生成 Java 程序文档。

输入下面给出的 Java 应用程序，利用 javadoc 命令生成该 Java 应用程序的文档，并使用浏览器 IE 显示生成的文档页面内容。

CommentToDoc.java

```
/* Java语言实验
   课程: Java语言
   时间: 2019-2020学年第一学期
*/
/**
   这是一个 Java语言实验程序，定义类 CommentToDoc。其中含有 main() 方法，故可以作为一个
   应用程序单独运行。其功能是在默认的输出设备上输出字符串"我在学习 Java语言"。
*/
public class CommentToDoc {
    //主方法，作为 Java应用程序的默认入口。
    public static void main(String args[ ]) {
        System.out.println("我在学习 Java语言"); //输出"我在学习 Java语言"
    }
}
```

执行指令：

`javadoc CommentToDoc.java`

生成文件：

- CommentTest.html
- package-frame.html
- package-summary.html
- package-tree.html
- constant-values.html
- overview-tree.html
- index-all.html
- deprecated-list.html
- allclasses-frame.html
- allclasses-noframe.html
- index.html
- help-doc.html

实验3 简单型变量输入输出

3.1 实验目的

掌握从键盘为简单型变量输入数据。掌握使用 Scanner 类创建一个对象，例如：

```
Scanner reader = new Scanner (System.in);
```

练习让 reader 对象调用下列方法读取用户在命令行（例如，MS-DOS 窗口）输入的各种简单类型数据：

- nextBoolean()
- nextByte()
- nextShort()
- nextInt()
- nextLong()
- nextFloat()
- nextDouble()

在调试程序时，体会上述方法都会堵塞，即程序等待用户在命令行输入数据回车确认。

3.2 实验要求

编写一个 Java 应用程序，在主类的 main 方法中声明用于存放产品数量的 int 型变量 amount 和产品单价的 float 型变量 price，以及存放全部产品总价值 float 型变量 sum。

使用 Scanner 对象调用方法让用户从键盘为 amount，price 变量输入数值，然后程序计算出全部产品总价值，并输出 amount，price 和 sum 的值。

3.3 程序模版

请按模板要求，将【代码】替换为 Java 程序代码。

InputData.java

```
import java.util.Scanner;
public class InputData {
    public static void main(String args[]) {
        Scanner reader=new Scanner(System.in);
        int amount=0;
        float price=0,sum=0;
        System.out.println("输入产品数量(回车确认):");
        【代码 1】    //从键盘为 amount赋值
        System.out.println("输入产品单价(回车确认):");
        【代码 2】    //从键盘为 price赋值
        sum = price*amount;
        System.out.printf("数量:%d,单价:%5.2f,总价值:%5.2f",amount,price,sum);
    }
}
```

3.4 实验指导

由于 amount 是 int 型，因此【代码 1】应该是 amount = reader.nextInt()，而 price 是 float 型，因此代码 2 应该是 price = reader.nextFloat()，不可以是 price = reader.nextDouble()。

另外，Scanner 对象可以调用 hasNextXXX()方法判断用户输入的数据的类型，例如，如果用户在键盘输入带小数点的数字：12.34（回车），那么 reader 对象调用 hasNextDouble()返回的值是 true，而调用 hasNextByte()、hasNextInt()以及 hasNextLong()返回的值都是 false；如果用户在键盘输入一个 byte 取值范围内的整数：89（回车），那么 reader 对象调用 hasNextByte()、hasNextInt()、hasNextLong()以及 hasNextDouble()返回的值都是 true。nextLine()等待用户在命令行输入一行文本回车，该方法得到一个 String 类型的数据，String 类型将在后续课程中讲述。

在从键盘输入数据时，我们经常让 reader 对象先调用 hasNextXXX()方法等待用户在键盘输入数据，然后再调用 nextXXX()方法读取数据。

3.5 扩展练习

3.5.1 反复输入实验

上机调试下列程序。该程序可以让用户在键盘依次输入若干个数字，每输入一个数字都需要按回车键确认，最后用户在键盘输入一个非数字字符串结束整个输入操作过程（用户输入非数字数字后 reader.hasNextDouble()的值将是 false）。程序将计算出这些数的和以及平均值。

MultipleInput.java

```
import java.util.*;
public class MultipleInput{
    public static void main (String args[ ]){
        Scanner reader = new Scanner(System.in);
        double sum = 0;
        int m = 0;
        while(reader.hasNextDouble()){
            double x = reader.nextDouble();
            m = m + 1;
            sum = sum + x;
        }
        System.out.printf("%d个数的和为%f\n",m,sum);
        System.out.printf("%d个数的平均值是%f\n",m,sum/m);
    }
}
```

3.5.2 转义字符实验

输入、运行以下 Java 应用程序，写出运行结果，以验证 Java 语言转义字符的功能。

TransChar.java

```
public class TransChar{
    public static void main(String args[ ]) {
        char ch1 = '\b';
        char ch2 = '\t';
        char ch3 = '\n';
        char ch4 = '\r';
        char ch5 = '\"';
        char ch6 = '\\';
        char ch7 = '\\';
        System.out.println("广州大学"+ch1+"计算机学院");
    }
}
```

```

        System.out.println("广州大学"+ch2+"计算机学院");
        System.out.println("广州大学"+ch3+"计算机学院");
        System.out.println("广州大学"+ch4+ch3+"计算机学院");
        System.out.println(ch5+"广州大学"+"计算机学院"+ch5);
        System.out.println(ch6+"广州大学"+"计算机学院"+ch6);
        System.out.println(ch7+"广州大学"+"计算机学院"+ch7);
    }
}

```

运行结果：

- 广州大 计算机学院
- 广州大学 计算机学院
- 广州大学
- 计算机学院
- 广州大学
- 计算机学院
- "广州大学计算机学院"
- '广州大学计算机学院'
- \广州大学计算机学院\

3.5.3 输入数据处理实验

输入、运行以下 Java 应用程序，写出运行结果，并说明程序所完成的功能。

ParseDouble.java

```

import java.io.*;
public class ParseDouble {
    public static void main(String args[]) {
        String s;
        double d;
        int i;
        boolean b = false;
        do {
            try {
                System.out.println("请输入一个浮点数：");
                BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
                s = br.readLine();           // 以字符串方式读入
                i = s.indexOf('.');          // 找到小数点的位置
                d = Double.parseDouble(s);   // 将字符串转换成浮点数
                System.out.println(d + " 整数部分为：" + (long)d);
                if(i == -1)                   // 若没有小数点，则没有小数部分
                    System.out.println(d + " 小数部分为：0.0");
                else                           // 若有小数点，则截取小数点后的字符串合成浮点数
                    System.out.println(d +
                        " 小数部分为：" +
                        Double.parseDouble(((s.charAt(0)=='-') ? "-" : "") +
                        "0." +
                        s.substring(i+1,s.length())));
                b = false;
            }
            catch(NumberFormatException nfe) {
                System.out.println("输入浮点数格式有误。\\n");
                b = true;
            }
        }
        catch(IOException ioe) {

```

```
        b = false;
    }
}
while(b);    // 浮点格式错误时重新输入
}           // end of main
}           // end of class
```

运行结果：

- 请输入一个浮点数：
- abc
- 输入浮点数格式有误
- 请输入一个浮点数：
- 3.1415926
- 3.1415926 的整数部分为：3
- 3.1415926 的小数部分为：0.1415926

实验4 分支和循环语句

4.1 实验目的

本实验的目的是熟悉如何使用循环语句解决问题。

4.2 实验要求

编写一个 Java 应用程序，输出区间[200, 300]上的所有素数，要求写出程序的运行结果。

4.3 程序模版

请按模板要求，将【代码】替换为 Java 程序代码。

FindPrime.java

```
public class FindPrime{
    public static void main(String[] args){
        int i,j;
        boolean isPrime;
        for(i=200;i<=300;i++){
            isPrime=true;
            for(j=2;j<i-1;j++){
                if(i%j==0){
                    【代码】
                }
            }
            if(isPrime){
                System.out.print(i+" ");
            }
        }
        System.out.println();
    }
}
```

运行结果：

- 211 223 227 229 233 239 241 251 257 263 269 271 277 281 283 293

4.4 实验指导

使用 break 语句可以跳出当前循环。

4.5 扩展练习

4.5.1 循环标签

continue 语句默认是结束当前循环的单个循环。有时我们在使用循环时，想通过内层循环里的语句直接结束外部循环的当前单个循环。Java 提供了相关的功能，只需要在 continue 后通过标签指定外层循环。Java 中的标签是一个紧跟着英文冒号的标识符，该标签只有放在循环语句之前才有作用。类似的，break 语句默认是结束当前循环，在使用循环时，也可以通过内层循环里中的 break 后指定外层循环标签，从而直接跳出外层循环。

FindPrime2.java

```
public class FindPrime2{
```

```
public static void main(String[] args){
    int i,j;
    outer:
    for(i=200;i<=300;i++){
        for(j=2;j<i-1;j++){
            if(i%j==0)
                continue outer;
        }
        System.out.print(i+" ");
    }
}
```

实验5 类与对象

5.1 实验目的

- 掌握 Java 类的定义和使用，以及对象的声明和使用。
- 理解构造函数的概念和使用方法。
- 掌握类及其成员的访问控制符的使用。

5.2 实验要求

按要求编写一个 Java 应用程序：第一，定义一个表示学生的类 Student。这个类的属性有“学号”、“班号”、“姓名”、“性别”、“年龄”，方法有“获得学号”、“获得班号”、“获得性别”、“获得姓名”、“获得年龄”、“获得年龄”；第二，为类 Student 增加一个方法 public String toString()，该方法把 Student 类的对象的所有属性信息组合成一个字符串以便输出显示。

5.3 程序模版

请按模版要求，将【代码】替换为 Java 程序代码。

StudentDemo.java

```
class Student{
    private long studentID;
    private int classID;
    private String name;
    private String sex;
    private int age;
    public Student(long studentID, int classID, String name,
                   String sex, int age){

        【代码】
    }
    public long getStudentID(){
        return studentID;
    }
    public int getClassID(){
        return classID;
    }
    public String getName(){
        return name;
    }
    public String getSex(){
        return sex;
    }
    public int getAge(){
        return age;
    }
    public String toString(){
        return "学号: "+getStudentID()+
               "\n班号: "+getClassID()+
               "\n姓名: "+getName()+
               "\n性别: "+getSex()+
               "\n年龄: "+getAge();
    }
}

public class StudentDemo{
    public static void main(String[] args){
```



```

        Student s1=new Student(90221,2,"Tom","male",20);
        System.out.println(s1.toString());
    }
}

```

运行结果是：

```

学号：90221
班号：2
姓名：Tom
性别：male
年龄：20

```

5.4 实验指导

在 Student 类的构造方法中，参数中的局部变量的名字与成员变量的名字相同，于是 Student 类的成员变量被隐藏，此时若想使用被隐藏的成员变量，必须使用关键字 this。

5.5 扩展练习

5.5.1 思考

对于上述实验中的 Student 类，尝试能否使用以下语句创建一个新的对象。如果不能，为什么？

```
Student s1=new Student();
```

5.5.2 拓展 STUDENT 类

拓展上述实验中的 Student 类，添加以下方法：“设置学号”、“设置班号”、“设置性别”、“设置姓名”、“设置年龄”、“设置年龄”，以及没有参数的构造方法。

StudentExtendedDemo.java

```

class StudentExtended{
    private long studentID;
    private int classID;
    private String name;
    private String sex;
    private int age;
    public Student(long studentID, int classID, String name,
        String sex, int age){
        【代码 1】
    }
    public long getStudentID(){
        return studentID;
    }
    public int getClassID(){
        return classID;
    }
    public String getName(){
        return name;
    }
    public String getSex(){
        return sex;
    }
    public int getAge(){

```

```

        return age;
    }
    public void setStudentID(long studentID){
        【代码 2】
    }
    public void setClassID(int classID){
        【代码 3】
    }
    public void setName(String name){
        【代码 4】
    }
    public void setSex(String sex){
        【代码 5】
    }
    public void setAge(int age){
        【代码 6】
    }
    public Student(){
    }
    public String toString(){
        return "学号: "+getStudentID()+
            "\n班号: "+getClassID()+
            "\n姓名: "+getName()+
            "\n性别: "+getSex()+
            "\n年龄: "+getAge();
    }
}
public class StudentExtendedDemo{
    public static void main(String[] args){
        Student s1=new Student(90221,2,"Tom","male",20);
        Student s2=new Student();
        【代码 7】
        System.out.println(s1.toString());
        System.out.println(s2.toString());
    }
}

```

要求运行结果是：

学号：90221

班号：2

姓名：Tom

性别：male

年龄：20

学号：90222

班号：1

姓名：Jerry

性别：female

年龄：21

实验6 类变量与实例变量

6.1 实验目的

类变量是与类相关联的数据变量，而实例变量是仅仅和对象相关联的数据变量。不同的对象的实例变量将被分配不同的内存空间，如果类中有类变量，那么所有对象的这个类变量都分配给相同的一处内存，改变其中一个对象的这个类变量会影响其他对象的这个类变量。也就是说，对象共享类变量。类中的方法可以操作成员变量，当对象调用方法时，方法中出现的成员变量就是指分配给该对象的变量，方法中出现的类变量也是该对象的变量，只不过这个变量和所有的其他对象共享而已。实例方法可操作实例成员变量和静态成员变量，静态方法只能操作静态成员变量。本实验的目的是掌握类变量与实例变量，以及类方法与实例方法的区别。

6.2 实验要求

编写程序模拟两个村庄共同拥有一片森林。编写一个 Village 类，该类有一个静态的 int 型成员变量 treeAmount 用于模拟森林中树木的数量。在主类 MainClass 的 main 方法中创建两个村庄，一个村庄改变了 treeAmount 的值，另一个村庄查看 treeAmount 的值。

6.3 程序模版

请按模版要求，将【代码】替换为 Java 程序代码。

Village.java

```
class Village {
    static int treeAmount; //模拟森林中树木的数量
    int peopleNumber;      //村庄的人数
    String name;           //村庄的名字
    Village(String s) {
        name = s;
    }
    void treePlanting(int n){
        treeAmount = treeAmount+n;
        System.out.println(name+"植树"+n+"颗");
    }
    void fellTree(int n){
        if(treeAmount-n>=0){
            treeAmount = treeAmount-n;
            System.out.println(name+"伐树"+n+"颗");
        }
        else {
            System.out.println("无树木可伐");
        }
    }
    static int lookTreeAmount() {
        return treeAmount;
    }
    void addPeopleNumber(int n) {
        peopleNumber = peopleNumber+n;
        System.out.println(name+"增加了"+n+"人");
    }
}
```

MainClass.java

```
public class MainClass {
    public static void main(String args[]) {
```

```

Village zhaoZhuang,maJiaHeZhi;
zhaoZhuang = new Village("赵庄");
maJiaHeZhi = new Village("马家河子");
zhaoZhuang.peopleNumber=100;
maJiaHeZhi.peopleNumber=150;
    【代码 1】 //用类名 Village 访问 treeAmount,并赋值 200
int leftTree =Village.treeAmount;
System.out.println("森林中有 "+leftTree+" 颗树");
    【代码 2】 //zhaoZhuang调用 treePlanting(int n),并向参数传值 50
leftTree = 【代码 3】 //maJiaHeZhi调用 lookTreeAmount()方法得到树木的数量
System.out.println("森林中有 "+leftTree+" 颗树");
    【代码 4】 //maJiaHeZhi调用 fellTree(int n),并向参数传值 70
leftTree = Village.lookTreeAmount();
System.out.println("森林中有 "+leftTree+" 颗树");
System.out.println("赵庄的人口:"+zhaoZhuang.peopleNumber);
zhaoZhuang.addPeopleNumber(12);
System.out.println("赵庄的人口:"+zhaoZhuang.peopleNumber);
System.out.println("马家河子的人口:"+maJiaHeZhi.peopleNumber);
maJiaHeZhi.addPeopleNumber(10);
System.out.println("马家河子的人口:"+maJiaHeZhi.peopleNumber);
}
}

```

6.4 实验指导

对象共享类变量，在代码 1 之前已经有了 zhaoZhuang 对象，这个时候，代码 1 用 Village.treeAmount=200 或 zhaoZhuang.treeAmount=200 替换都是正确的。

6.5 扩展练习

代码 2 是否可以写成 "Village.treePlanting(50);" ？

实验7 子类与继承

7.1 实验目的

上转型对象可以访问子类继承或隐藏的成员变量，也可以调用子类继承的方法或子类重写的实例方法。上转型对象操作子类继承的方法或子类重写的实例方法，其作用等价于子类对象去调用这些方法。因此，如果子类重写了父类的某个实例方法后，当对象的上转型对象调用这个实例方法时一定是调用子类重写的实例方法。本实验的目的是掌握上转型对象的使用，理解不同对象的上转型对象调用同一方法可能产生不同的行为，即理解上转型对象在调用方法时可能具有多种形态。

7.2 实验要求

- 1、编写一个 abstract 类，类名为 Geometry，该类有一个 abstract 方法。
- 2、编写 Geometry 的若干个子类，比如 Circle 子类和 Rect 子类。
- 3、编写 Student 类，该类定义一个 public double area(Geometry...p)方法，该方法的参数是可变参数，即参数的个数不确定，但类型都是 Geometry。该方法返回参数计算的面积之和。
- 4、在主类 MainClass 的 main 方法中创建一个 Student 对象，让该对象调用 public double area(Geometry...p)计算若干个矩形和若干个圆的面积之和。

7.3 程序模版

请按模板要求，将【代码】替换为 Java 程序代码。

Geometry.java

```
public abstract class Geometry {  
    public abstract double getArea();  
}
```

Rect.java

```
public class Rect extends Geometry {  
    double a, b;  
    Rect(double a, double b) {  
        this.a = a;  
        this.b = b;  
    }  
    【代码1】 //重写 getArea()方法,返回矩形面积  
}
```

Circle.java

```
public class Circle extends Geometry {  
    double r;  
    Circle(double r) {  
        this.r = r;  
    }  
    【代码2】 //重写 getArea()方法,返回圆面积  
}
```

Student.java

```
public class Student {  
    public double area(Geometry ...p) {  
        double sum=0;  
        for(int i=0; i<p.length; i++) {  
            sum = sum + p[i].getArea();  
        }  
    }  
}
```

```

        return sum;
    }
}
MainClass.java
public class MainClass {
    public static void main(String args[]) {
        Student zhang = new Student();
        double area =
            zhang.area(new Rect(2,3), new Circle(5.2), new Circle(12));
        System.out.printf("2个圆和 1个矩形图形的面积和: \n%10.3f",area);
    }
}

```

7.4 实验指导

尽管 abstract 类不能创建对象，但 abstract 类声明的对象可以存放子类对象的引用，即成为子类对象的上转型对象。由于 abstract 类可以有 abstract 方法，这样就保证子类必须要重写这些 abstract 方法。由于 Student 类的 area 方法的参数类型是 Geometry，因此可以将其子类的对象的引用传递给方法 area 的参数。因此 area 方法可以通过循环语句让每个参数调用 getArea 方法，计算各自的面积。

可变参数是 JDK1.5 版本后新增的功能。可变参数声明方法时不给出参数列表中从某项直至最后一项参数的名字和个数，但这些参数的类型必须相同。可变参数使用"..."表示若干个参数，这些参数的类型必须相同，最后一个参数必须是参数列表中的最后一个参数。例如：

```
public void f(int ... x)
```

那么，方法 f 的参数列表中，从第一个至最后一个参数都是 int 型，但连续出现的 int 型参数的个数不确定。称 x 是方法 f 的参数列表中的可变参数的“参数代表”。参数代表可以借助下标运算来表示参数列表中的具体参数，即 x[0], x[1], ..., x[m] 分别表示 x 代表的第 1 个至 m+1 个参数。

对于 Student 类中的 area(Geometry...p) 方法中的可变参数 p，那么 p[i] 就是第 i+1 个参数。

7.5 扩展练习

7.5.1 编写 TRIANGLE 类

编写一个 Geometry 的子类 Triangle，可以计算三角形的面积，在主类中让 Student 类的对象 zhang 调用 area 方法计算 1 个三角形和 2 个圆的面积之和。

7.5.2 分析类的关系

阅读如下所示的 3 个 Java 类的定义，分析它们之间的关系，写出运行结果

```

class SuperClass {
    int x;
    SuperClass() {
        x=3;
        System.out.println("in SuperClass : x=" +x);
    }
    void doSomething() {
        System.out.println("in SuperClass.doSomething()");
    }
}

```

```

class SubClass extends SuperClass {
    int x;
    SubClass() {
        super();        //调用父类的构造方法
        x=5;            //super( ) 要放在方法中的第一句
        System.out.println("in SubClass :x="+x);
    }
    void doSomething( ) {
        super.doSomething( ); //调用父类的方法
        System.out.println("in SubClass.doSomething()");
        System.out.println("super.x="+super.x+" sub.x="+x);
    }
}
public class Inheritance {
    public static void main(String args[]) {
        SubClass subC=new SubClass();
        subC.doSomething();
    }
}

```

运行结果：

in SuperClass: x=3

in SubClass: x=5

in SuperClass.doSomething()

in SubClass.doSomething()

super.x=3 sub.x=5

7.5.3 类的设计

请按以下要求，完成类的设计，并编写 Java 应用程序：假定根据学生的 3 门学位课程的分
数决定其是否可以拿到学位。对于本科生，如果 3 门课程的平均分数超过 60 分即表示通过；而
对于研究生，则需要平均超过 80 分才能够通过。

- 设计一个基类 Student 描述学生的共同特征。
- 设计一个描述本科生的类 Undergraduate，该类继承并扩展 Student 类。
- 设计一个描述研究生的类 Graduate，该类继承并扩展 Student 类。
- 设计一个测试类 StudentDemo，分别创建本科生和研究生这两个类的对象，并输出相
关信息。

StudentDemo2.java

```

class Student{
    private String name;
    private int classA,classB,classC;
    public Student(String name, int classA, int classB, int classC){
        this.name=name;
        this.classA=classA;
        this.classB=classB;
        this.classC=classC;
    }
    public String getName(){
        return name;
    }
    public int getAverage(){
        return (classA+classB+classC)/3;
    }
}

```

```

    }
}
class UnderGraduate extends Student{
    public UnderGraduate(String name,int classA,int classB,int classC){
        super(name,classA,classB,classC);
    }
    public void isPass(){
        if(getAverage()>=60)
            System.out.println("本科生"+getName()+
                                "的三科平均分为: "+getAverage()+
                                ",可以拿到学士学位。");
        else
            System.out.println("本科生"+getName()+
                                "的三科平均分为: "+getAverage()+
                                ",不能拿到学士学位。");
    }
}
class Graduate extends Student{
    public Graduate(String name,int classA,int classB,int classC){
        super(name,classA,classB,classC);
    }
    public void isPass(){
        if(getAverage()>=80)
            System.out.println("研究生"+getName()+
                                "的三科平均分为: "+getAverage()+
                                ",可以拿到硕士学位。");
        else
            System.out.println("研究生"+getName()+
                                "的三科平均分为: "+getAverage()+
                                ",不能拿到硕士学位。");
    }
}
}
public class StudentDemo2{
    public static void main(String[] args){
        UnderGraduate s1=new UnderGraduate("Tom",55,75,81);
        Graduate s2=new Graduate("Mary",72,81,68);
        s1.isPass();
        s2.isPass();
    }
}

```

运行结果：

本科生 Tom 的三科平均分为：70，可以拿到学士学位。

研究生 Mary 的三科平均分为：73，不能拿到硕士学位。

实验8 文件读写

8.1 实验目的

- 掌握 Java I/O 基本原理。
- 掌握 InputStream、OutputStream 抽象类的基本使用方法。
- 掌握 FileInputStream、FileOutputStream 抽象类的基本使用方法。

8.2 实验要求

输入、运行以下 Java 应用程序，写出运行结果，理解读取文件的基本方法。

8.3 程序模版

ReadFile.java

```
import java.io.*;
public class ReadFile{
    public static void main(String[] args) throws IOException{
        BufferedReader br=new BufferedReader(new FileReader(
            "ReadFile.java"));
        String s=br.readLine();
        while(s!=null){
            System.out.println(s);
            s=br.readLine();
        }
        br.close();
    }
}
```

输入、运行以下 Java 应用程序，写出运行结果，理解读写文件的基本方法。

CopyFile.java

```
import java.io.*;
public class CopyFile {
    public static void main(String[] args) {
        try {
            FileInputStream fis = new FileInputStream("CopyFile.java");
            FileOutputStream fos = new FileOutputStream("temp.txt");
            int read = fis.read();
            while ( read != -1 ) {
                fos.write(read);
                read = fis.read();
            }
            fis.close();
            fos.close();
        }
        catch (IOException e) {
            System.out.println(e);
        }
    }
}
```

8.4 实验指导

CopyFile 类的功能是完成文件的复制：通过字节流从 “CopyFile.java” 文件中读取数据并写入到 “temp.txt” 文件中，实现文件复制功能。

8.5 扩展练习

编写一个 Java 应用程序，实现如下的设计功能：运行该程序可以列出当前目录下的文件。

```
import java.io.*;
public class FileList2{
    public static void main(String[] args){
        File dir=new File(".");
        File files[]=dir.listFiles();
        System.out.println(dir);
        for(int i=0;i<files.length;i++){
            if(files[i].isFile())
                System.out.println("\t"+files[i].getName());
            else
                System.out.println("<DIR>\t"+files[i].getName());
        }
    }
}
```

实验9 面向接口编程

9.1 实验目的

接口回调是多态的另一种体现，接口回调是指可以把使用某一接口的类创建的对象引用赋值给该接口声明的接口变量中，那么该接口变量就可以调用被类实现的接口中的方法，当接口变量调用被类实现的接口中的方法时，就是通知相应的对象调用接口的方法，这一过程成为对象功能的接口回调。所谓面向接口编程，是指当设计某种重要的类时，不让该类面向具体的类，而是面向接口，即设计类中的重要数据是接口声明的变量，而不是具体类声明的对象。本实验的目的是掌握接口回调和面向接口编程思想。

9.2 实验要求

小狗在不同环境条件下可能呈现不同的状态，小狗通过调用 cry()方法体现自己的当前的状态。要求用接口封装小狗的状态。具体要求如下。

- 编写一个接口 DogState，该接口有一个名字为 void showState()的方法。
- 编写 Dog 类，该类中有一个 DogState 接口声明的变量 state。另外，该类有一个 cry()方法，在该方法中让接口 state 回调 showState()方法。即 Dog 对象通过 cry()方法来体现自己目前的状态。
- 编写若干个实现 DogState 接口的类，负责刻画小狗的各种状态。
- 编写主类，在主类中用 Dog 创建小狗，并让小狗调用 cry 方法体现自己的状态。

9.3 程序模板

请按模板要求，将【代码】替换为 Java 程序代码。

E.java

```
interface DogState {
    public void showState();
}
class SoftlyState implements DogState {
    【代码1】 //重写 public void showState()
}
class MeetEnemyState implements DogState {
    【代码2】 //重写 public void showState()
}
class MeetFriendState implements DogState {
    【代码3】 //重写 public void showState()
}
class Dog {
    DogState state;
    public void cry() {
        state.showState();
    }
    public void setState(DogState s) {
        state = s;
    }
}
public class E {
    public static void main(String args[]) {
        Dog yellowDog =new Dog();
        yellowDog.setState(new SoftlyState());
    }
}
```

```

        yellowDog.cry();
        yellowDog.setState(new MeetEnemyState());
        yellowDog.cry();
        yellowDog.setState(new MeetFriendState());
        yellowDog.cry();
    }
}

```

9.4 实验指导

接口 DogState 规定了状态的方法名称，因此，实现该接口的类，例如 MeetEnemyState 类，必须具体实现接口中的方法 public void showState()，以便体现小狗遇到敌人时是怎样的状态，例如，程序中的代码 2 可以是：

```

public void showState() {
    System.out.println ("遇到敌人狂叫，并冲向去很咬敌人");
}

```

9.5 扩展练习

由于 Dog 类是面向 DogState 接口，并让小狗通过 cry 方法来体现自己的状态，因此当再增加一个实现 DogState 接口的类后(即给小狗增加一种状态)，Dog 类不需要进行修改。

请增加如下的类。

```

class MeetAnotherDog implements DogState {
    public void showState() {
        System.out.println("嬉戏");
    }
}

```

并在主类中测试小狗遇到同类时调用 cry()的效果。

实验10 实验报告（一） 题目

10.1 实验目的

- 掌握开发 Java 应用程序的步骤，掌握 Java 应用程序的基本结构。
- 掌握 Java 基本数据类型在命令行的输入和输出方法。
- 熟悉如何使用 Java 分支和循环语句解决问题。
- 熟悉类的基本设计方法，根据 Java 类的继承机制有效解决问题。

10.2 实验任务 1

编写一个 Java 应用程序，在主类的 main 方法中实现下列功能。

- 程序随机分配给用户一个 1 至 100 之间的整数
- 用户通过键盘输入自己的猜测
- 程序返回提示信息，提示信息分别是：“猜大了”、“猜小了”和“猜对了”
- 用户可根据提示信息再次输入猜测，直到提示信息是“猜对了”
- 用户猜对以后，显示猜测次数，并提供“重新开始”和“退出”功能

10.3 实验任务 2

假定要为某个公司编写雇员工资支付程序，这个公司有各种类型的雇员（Employee），不同类型的雇员按不同的方式支付工资：

- 经理（Manager）：每月获得一份固定的工资
- 销售人员（Salesman）：在基本工资的基础上每月还有销售提成
- 工人（Worker）：按照每月工作的天数计算工资

根据上述要求试用类的继承和相关机制描述这些功能，并编写一个 Java 应用程序，演示这些类的用法。

提示：应设计一个雇员类（Employee）描述所有雇员的共同特性，这个类应该提供一个计算工资的抽象方法 `ComputeSalary()`，使得可以通过这个类计算所有雇员的工资。经理、销售人员和一般工人对应的类都应该继承这个类，并重新定义计算工资的方法，进而给出它的具体实现。

第二部分

JAVA 图形图像和多线程应用

实验11 JAVA 图形界面的组件与布局

11.1 实验目的

- 了解 Java 系统图形用户界面的工作原理和界面设计步骤。
- 掌握图形用户界面的各种常用组件的使用方法。
- 掌握图形用户界面各种布局策略的设计与使用。

11.2 实验要求

编写一个 Java 应用程序，实现以下要求：显示一个 100*100 的窗口，窗口内添加了四个按钮，其布局为流式布局管理器。当窗口 f 的尺寸被重置后，其 FlowLayout 型的布局也会随之发生变化，各按钮的大小不变，但其相对位置会变化。



11.3 程序模版

请按模板要求，将【代码】替换为 java 程序代码。

TestFlowLayout.java

```
import javax.swing.*;
import java.awt.*;
public class TestFlowLayout {
    public static void main(String args[]) {
        JFrame f = new JFrame("Flow Layout");
        JButton button1 = new JButton("确定");
        JButton button2 = new JButton("打开");
        JButton button3 = new JButton("关闭");
        JButton button4 = new JButton("取消");
        f.setLayout(new FlowLayout());
        【代码 1】 // 在窗口中添加 button1
        【代码 2】 // 在窗口中添加 button2
        【代码 3】 // 在窗口中添加 button3
        【代码 4】 // 在窗口中添加 button4
        f.setSize(100,100);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

11.4 实验指导

使用 add()方法可以在窗口中添加组建。例如：

```
f.add(button1);
```

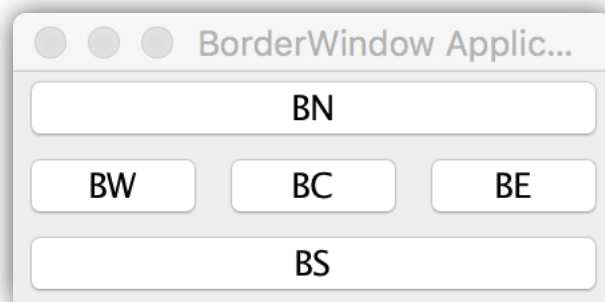
11.5 扩展练习

11.5.1 常用布局

当把组件添加到容器中时，希望控制组件在容器中的位置，这就需要布局设计。上面的实验中使用了 FlowLayout 布局，其他常见的布局还有 BorderLayout，CardLayout，GridLayout 等，修改上述实验代码，尝试其他布局方式。

11.5.2 BORDERLAYOUT

BorderLayout 布局将容器空间简单地划分为东、西、南、北、中 5 个区域，每加入一个组件都应该指明把这个组件加在哪个区域中。区域由 BorderLayout 中的常量 NORTH，SOUTH，EAST，WEST，CENTER 表示。编写一个 Java 应用程序，实现以下窗口布局。

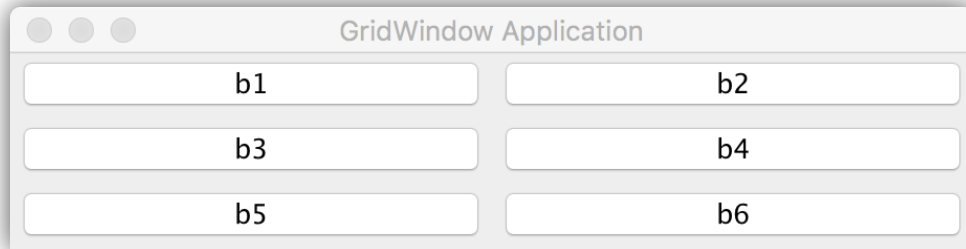


BorderLayoutWindow.java

```
import javax.swing.*;
import java.awt.*;
public class BorderLayoutWindow extends JFrame {
    public BorderLayoutWindow() {
        setLayout(new BorderLayout());
        add(new JButton("BN"),BorderLayout.NORTH);
        add(new JButton("BS"),BorderLayout.SOUTH);
        add(new JButton("BE"),BorderLayout.EAST);
        add(new JButton("BW"),BorderLayout.WEST);
        add(new JButton("BC"),BorderLayout.CENTER);
    }
    public static void main(String args[]) {
        BorderLayoutWindow window = new BorderLayoutWindow();
        window.setTitle("BorderWindow Application");
        window.pack();
        window.setVisible(true);
        window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

11.5.3 GRIDLAYOUT

GridLayout 将容器划分为若干行*若干列的网格区域，组件就放置在这些划分出来的单元格中。使用 GridLayout 的构造方法 GridLayout(int m, int n)创建布局对象，可以制定划分网格的行数 m 和列数 n。编写一个 Java 应用程序，实现以下窗口布局。



GridLayoutWindow.java

```
import javax.swing.*;
import java.awt.*;
public class GridLayoutWindow extends JFrame {
    public GridLayoutWindow() {
        setLayout(new GridLayout(3,2));
        add(new JButton("b1"));
        add(new JButton("b2"));
        add(new JButton("b3"));
        add(new JButton("b4"));
        add(new JButton("b5"));
        add(new JButton("b6"));
    }
    public static void main(String args[]) {
        GridLayoutWindow window = new GridLayoutWindow();
        window.setTitle("GridWindow Application");
        window.pack();
        window.setVisible(true);
        window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

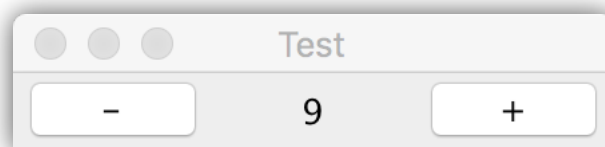

实验12 事件处理

12.1 实验目的

- 了解 Java 图形用户界面的事件响应机制。
- 掌握鼠标事件编程方法。
- 掌握 AWT 中 Color 和 Font 类的使用方法。

12.2 实验要求

编写一个 Java 应用程序，实现以下要求：创建一个 GridLayout 的框架 f，其标题为 Test。在框架中添加了一个标签为“-”的按钮 b1，一个标签，以及一个标签为“+”的按钮 b2。标签中显示一个数值，初始化为 0。为按钮 b1 和 b2 注册监听器 bh，监听 ActionEvent 事件，当单击框架中的按钮时，会触发 ActionEvent 事件，执行事件处理器 actionPerformed(ActionEvent e)。当点击 b1 按钮则标签中的数值减 1（减到 0 为止），当点击 b2 按钮则标签中的数值加 1。



12.3 程序模版

请按模板要求，将【代码】替换为 java 程序代码。

TestActionEvent.java

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class TestActionEvent {
    public static void main(String args[]) {
        JFrame f = new JFrame("Test");
        JButton b1 = new JButton("-");
        JButton b2 = new JButton("+");
        JLabel l = new JLabel("0",JLabel.CENTER);
        Monitor bh = new Monitor();
        f.setLayout(new GridLayout());
        bh.setJLabel(l);
        f.add(b1);
        f.add(l);
        f.add(b2);
        b1.setActionCommand("-");
        b2.setActionCommand("+");
        【代码 1】 // 为按钮 b1注册监听器 bh
        【代码 2】 // 为按钮 b2注册监听器 bh
        f.pack();
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
class Monitor implements ActionListener {
```

```

        int count = 0;
        JLabel l;
        public void setJLabel (JLabel l) {
            this.l = l;
        }
        public void actionPerformed(ActionEvent e) {
            String str = e.getActionCommand();
            if(str.equals("-")) {
                if(count>0) count--;
            }
            else {
                count++;
            }
            l.setText(""+count);
        }
    }
}

```

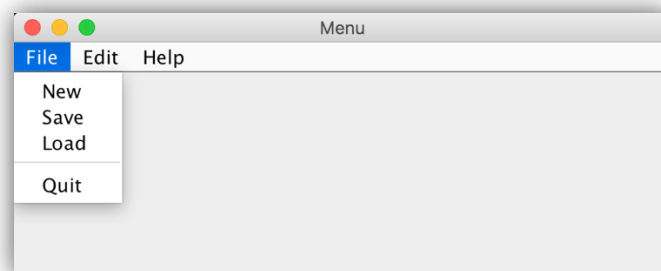
12.4 实验指导

为组件注册监听器可以使用以下语句：

```
addActionListener(监听器);
```

12.5 扩展练习

编写一个 Java 应用程序，实现菜单栏，程序运行的运行结果如下所示：



MenuTest.java

```

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class MenuTest{
    public static void main(String args[]){
        JFrame f = new JFrame("Menu");
        JMenuBar mb = new JMenuBar();
        f.setJMenuBar(mb);
        JMenu m1 = new JMenu("File");
        JMenu m2 = new JMenu("Edit");
        JMenu m3 = new JMenu("Help");
        mb.add(m1);
        mb.add(m2);
        mb.add(m3);
        JMenuItem m11 = new JMenuItem("New");
        JMenuItem m12 = new JMenuItem("Save");
        JMenuItem m13 = new JMenuItem("Load");
        JMenuItem m14 = new JMenuItem("Quit");
        m1.add(m11);
        m1.add(m12);
        m1.add(m13);
    }
}

```

```
        m1.addSeparator();
        m1.add(m14);
        f.pack();
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

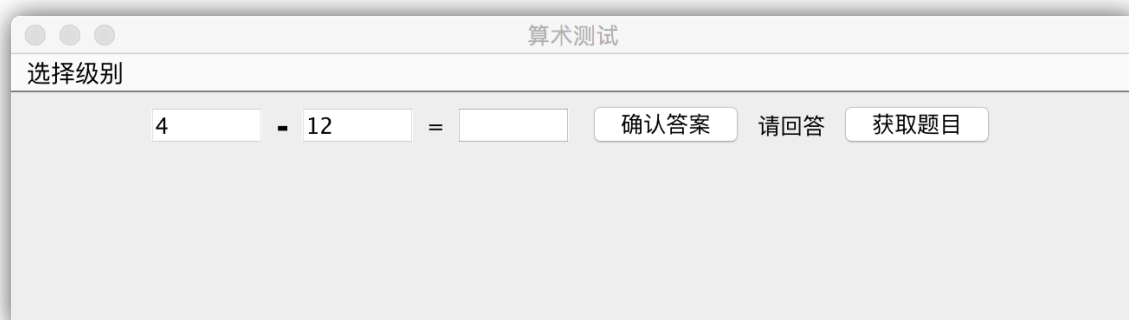
实验13 算术测试程序

13.1 实验目的

处理事件时，要很好地掌握事件源、监视器、处理事件的接口之间的关系。事件源是能够产生事件的对象，如文本框、按钮、下拉式列表等。事件源通过调用相应的方法将某个对象作为自己的监视器，事件源增加监视的方法 `addXXXListener(XXXListener listener)` 中的参数是一个接口，`listener` 可以引用任何实现了该接口的类创建的对象作为事件源的监视器，当事件源发生事件时，接口 `listener` 立刻调用被类实现的接口中的某个负责处理事件源发生的事件。本实验目的是掌握处理 `ActionEvent` 事件。

13.2 实验要求

编写一个算术测试小软件，用来训练小学生的算术能力。程序由 3 个类组成，其中 `Teacher` 对象充当监视器，负责给出算术题目，并判断回答者的答案是否正确。`ComputerFrame` 对象负责为算术题目提供视图，例如用户可以通过 `ComputerFrame` 对象提供的 GUI 界面看到题目，并通过该 GUI 界面给出题目的答案；`MainClass` 是软件的主类。程序运行效果如下图：



13.3 程序模版

请按模版要求，将【代码】替换为 Java 程序代码。

MainClass.java

```
public class MainClass {
    public static void main(String args[]) {
        ComputerFrame frame;
        frame=new ComputerFrame();
        frame.setTitle("算术测试");
        frame.setBounds(100,100,650,180);
    }
}
```

ComputerFrame.java

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class ComputerFrame extends JFrame {
    JMenuBar menubar;
    JMenu choiceGrade; //选择级别的菜单
    JMenuItem grade1,grade2;
    JTextField textOne,textTwo,textResult;
```

```

JButton getProblem,giveAnswer;
JLabel operatorLabel,message;
Teacher teacherZhang;
ComputerFrame() {
    teacherZhang=new Teacher();
    teacherZhang.setMaxInteger(20);
    setLayout(new FlowLayout());
    menubar = new JMenuBar();
    choiceGrade = new JMenu("选择级别");
    grade1 = new JMenuItem("幼儿级别");
    grade2 = new JMenuItem("儿童级别");
    grade1.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            teacherZhang.setMaxInteger(10);
        }
    });
    grade2.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            teacherZhang.setMaxInteger(50);
        }
    });

    choiceGrade.add(grade1);
    choiceGrade.add(grade2);
    menubar.add(choiceGrade);
    setJMenuBar(menubar);
    【代码 1】          //创建 textOne,其可见字符长是 5
    textTwo=new JTextField(5);
    textResult=new JTextField(5);
    operatorLabel=new JLabel("+");
    operatorLabel.setFont(new Font("Arial",Font.BOLD,20));
    message=new JLabel("你还没有回答呢");
    getProblem=new JButton("获取题目");
    giveAnswer=new JButton("确认答案");
    add(textOne);
    add(operatorLabel);
    add(textTwo);
    add(new JLabel("="));
    add(textResult);
    add(giveAnswer);
    add(message);
    add(getProblem);
    textResult.requestFocus();
    textOne.setEditable(false);
    textTwo.setEditable(false);
    getProblem.setActionCommand("getProblem");
    textResult.setActionCommand("answer");
    giveAnswer.setActionCommand("answer");
    teacherZhang.setJTextField(textOne,textTwo,textResult);
    teacherZhang.setJLabel(operatorLabel,message);
    【代码 2】//将 teacherZhang注册为 getProblem的 ActionEvent事件监视器
    【代码 3】//将 teacherZhang注册为 giveAnswer的 ActionEvent事件监视器
    【代码 4】//将 teacherZhang注册为 textResult的 ActionEvent事件监视器
    setVisible(true);
    validate();
    setDefaultCloseOperation(DISPOSE_ON_CLOSE);
}
}

```

Teacher.java

```

import java.util.Random;
import java.awt.event.*;

```

```

import javax.swing.*;
public class Teacher implements ActionListener {
    int numberOne,numberTwo;
    String operator="";
    boolean isRight;
    Random random;    //用于给出随机数
    int maxInteger;    //题目中最大的整数
    JTextField textOne,textTwo,textResult;
    JLabel operatorLabel,message;
    Teacher() {
        random = new Random();
    }
    public void setMaxInteger(int n) {
        maxInteger=n;
    }
    public void actionPerformed(ActionEvent e) {
        String str = e.getActionCommand();
        if(str.equals("getProblem")) {
            numberOne = random.nextInt(maxInteger)+1;//1至 maxInteger之间的随机数;
            numberTwo=random.nextInt(maxInteger)+1;
            double d=Math.random(); // 获取(0,1)之间的随机数
            if(d>=0.5)
                operator="+";
            else
                operator="-";
            textOne.setText(""+numberOne);
            textTwo.setText(""+numberTwo);
            operatorLabel.setText(operator);
            message.setText("请回答");
            textResult.setText(null);
        }
        else if(str.equals("answer")) {
            String answer=textResult.getText();
            try{
                int result=Integer.parseInt(answer);
                if(operator.equals("+")){
                    if(result==numberOne+numberTwo)
                        message.setText("你回答正确");
                    else
                        message.setText("你回答错误");
                }
                else if(operator.equals("-")){
                    if(result==numberOne-numberTwo)
                        message.setText("你回答正确");
                    else
                        message.setText("你回答错误");
                }
            }
            catch(NumberFormatException ex) {
                message.setText("请输入数字字符");
            }
        }
    }
    public void setJTextField(JTextField ... t) {
        textOne=t[0];
        textTwo=t[1];
        textResult=t[2];
    }
    public void setJLabel(JLabel ...label) {
        operatorLabel=label[0];
        message=label[1];
    }
}

```

```
}  
}
```

13.4 实验指导

需要将实验中的三个 java 文件保存在同一文件中，分别编译或只编译主类 MainClass，然后运行主类即可。JButton 对象可触发(ActionEvent) 事件，JButton 事件源使用 addActionListener 方法获得监视器，创建监视器的类需实现 ActionListener 接口。

13.5 扩展练习

给上述程序增加测试乘法的功能。

实验14 JAVA 异常类

14.1 实验目的

Java 实验 try-catch 语句来处理异常，将可能出现的异常操作放在 try-catch 语句的 try 部分，一旦 try 部分抛出异常对象，例如调用某个抛出异常的方法抛出了异常对象，那么 try 部分将立刻结束执行，而转向执行相应的 catch 部分。本实验的目的是掌握使用 try-catch 语句。

14.2 实验要求

车站检查危险品的设备，如果发现危险品会发出警告。编程模拟设备发现危险品。

- 编写一个 Exception 的子类 DangerException。该子类可以创建异常对象，该异常对象调用 toShow()方法输出"属于危险品"。
- 编写一个 Machine 类，该类的方法 checkBag(Goods goods)当发现参数 goods 是危险品时（即 goods 的 isDanger 属性的值是 true 时）将抛出 DangerException 异常。
- 程序在主类的 main 方法中的 try-catch 语句的 try 部分让 Machine 类的实例 checkBag(Goods goods)方法，一旦发现危险品，就在 try-catch 语句的 catch 部分处理危险品。

14.3 程序模板

请按模板要求，将【代码】替换为 java 程序代码。

check.java

```
class Goods {
    boolean isDanger;
    String name;
    Goods(String name) {
        this.name = name;
    }
    public void setIsDanger(boolean boo) {
        isDanger = boo;
    }
    public boolean isDanger() {
        return isDanger;
    }
    public String getName() {
        return name;
    }
}
class DangerException extends Exception {
    String message;
    public DangerException() {
        message = "危险品!";
    }
    public void toShow() {
        System.out.print(message+" ");
    }
}
class Machine {
    public void checkBag(Goods goods) throws DangerException {
        if(goods.isDanger()) {
            DangerException danger=new DangerException() ;
            【代码1】 //抛出 danger
        }
    }
}
```



```

    }
}
}
public class Check {
    public static void main(String args[]) {
        Machine machine = new Machine();
        Goods apple = new Goods("苹果");
        apple.setIsDanger(false);
        Goods explosive = new Goods("炸药");
        explosive.setIsDanger(true);
        try {
            machine.checkBag(explosive);
            System.out.println(explosive.getName()+"检查通过");
        }
        catch(DangerException e) {
            【代码 2】 //e调用 toShow()方法
            System.out.println(explosive.getName()+"被禁止!");
        }
        try {
            machine.checkBag(apple);
            System.out.println(apple.getName()+"检查通过");
        }
        catch(DangerException e) {
            e.toShow();
            System.out.println(apple.getName()+"被禁止!");
        }
    }
}
}

```

14.4 实验指导

throw 是 Java 的关键字，该关键字的作用就是抛出异常，因此代码 1 应该是 throw(danger)。

14.5 扩展练习

是否可以将实验代码里 try-catch 语句的 catch 部分捕获的 DangerException 异常更改为 Exception？

是否可以将实验代码里 try-catch 语句的 catch 部分捕获的 DangerException 异常更改为 java.io.IOException？

实验15 STRINGTOKENIZER 类

15.1 实验目的

当分析一个字符串并将字符串分解成可被独立使用的单词时，可以使用 java.util 包中 StringTokenizer 类。当我们想分解出字符串的有用的单词时，可以首先把字符串中不需要的单词都统一替换为空格或其他字符，例如"*"，然后再使用 StringTokenizer 类，并用"*"或空格做分隔标记分解出需要的单词。本实验的目的是掌握 StringTokenizer 类。

15.2 实验要求

两张购物小票的内容如下。

- "苹果 56.7 圆,香蕉:12 圆,芒果:19.8 圆";
- "酱油 6.7 圆,精盐:0.8 圆,榨菜:9.8 圆";

编写程序分别输出两张购物小票的价格之和。

15.3 程序模板

上机调试模板给出的程序，完成实验后的练习。

E.java

```
import java.util.*;
public class E {
    public static void main(String args[ ]) {
        String s1 = "苹果:56.7圆,香蕉:12圆,芒果:19.8圆";
        String s2 = "酱油:6.7圆,精盐:0.8圆,榨菜:9.8圆";
        ComputePice jisuan = new ComputePice();
        String regex = "[^0123456789.]+"; // 匹配所有非数字字符串
        String s1_number = s1.replaceAll(regex, "*");
        double priceSum = jisuan.compute(s1_number, "*");
        System.out.printf("\n%s\n价格总和:\n%f 圆\n", s1, priceSum);
        String s2_number = s2.replaceAll(regex, "#");
        priceSum = jisuan.compute(s2_number, "#");
        System.out.printf("\n%s\n价格总和:\n%f圆\n", s2, priceSum);
    }
}
class ComputePice {
    double compute(String s, String fenge) {
        StringTokenizer fenxiOne = new StringTokenizer(s, fenge);
        double sum = 0;
        double digitItem = 0;
        while(fenxiOne.hasMoreTokens()) {
            String str = fenxiOne.nextToken();
            digitItem = Double.parseDouble(str);
            sum = sum + digitItem;
        }
        return sum;
    }
}
```

15.4 实验指导

如果准备分解出"酱油:6.7 圆,精盐:0.8 圆,榨菜:9.8 圆"的货品名称, 即不要价格和价格单位以及标点符号, 那么可以实现使用正则表达式"[0123456789.]+圆"匹配诸如 ddddd.ddd 圆的价格数据。那么对于 String temp = s1.replaceAll(re,"");temp 就是字符串:

"酱油:,精盐:,榨菜:"

那么再经过:

```
temp = temp.replaceAll(":", " ");  
temp = temp.replaceAll(",", " ");
```

之后, temp 就是字符串:

"酱油 精盐 榨菜"

15.5 扩展练习

编写程序输出"酱油:6.7 圆,精盐:0.8 圆,榨菜:9.8 圆"中的货品名称。

实验16 输入输出流

16.1 实验目的

本实验的目的是掌握字符输入输出流以及缓冲输入输出流用法。

16.2 实验要求

现在有如下格式的成绩单（文本格式）score.txt。

姓名:张三,数学 72分,物理 67分,英语 70分.

姓名:李四,数学 92分,物理 98分,英语 88分.

姓名:周五,数学 68分,物理 80分,英语 77分.

要求按行读取成绩单，并在该行的后面加上该字的总成绩，然后将该行写入到一个名字为scoreAnalysis.txt 的文件中。

16.3 程序模版

请按模版要求，将【代码】替换为 Java 程序代码。

AnalysisResult.java

```
import java.io.*;
import java.util.*;
public class AnalysisResult {
    public static void main(String args[]) {
        File fRead = new File("score.txt");
        File fWrite = new File("socreAnalysis.txt");
        try{
            Writer out = 【代码 1】//以尾加方式创建指向文件 fWrite的 out流
            BufferedWriter bufferWrite = 【代码 2】//创建指向 out的 bufferWrite流
            Reader in = 【代码 3】//创建指向文件 fRead的 in流
            BufferedReader bufferRead = 【代码 4】//创建指向 in的 bufferRead流
            String str = null;
            while((str=bufferRead.readLine())!=null) {
                double totalScore=Fenxi.getTotalScore(str);
                str = str+" 总分:"+totalScore;
                System.out.println(str);
                bufferWrite.write(str);
                bufferWrite.newLine();
            }
            bufferRead.close();
            bufferWrite.close();
        }
        catch(IOException e) {
            System.out.println(e.toString());
        }
    }
}
```

Fenxi.java

```
import java.util.*;
public class Fenxi {
    public static double getTotalScore(String s) {
        Scanner scanner = new Scanner(s);
        scanner.useDelimiter("[^0123456789.]+");
        double totalScore=0;
        while(scanner.hasNext()){
```

```
        try{
            double score = scanner.nextDouble();
            totalScore = totalScore+score;
        }
        catch(InputMismatchException exp){
            String t = scanner.next();
        }
    }
    return totalScore;
}
}
```

16.4 实验指导

因为要以尾加方式创建指向文件 fWrite 的 out 流，即不刷新文件 scoreAnalysis.txt，因此代码 1 可以是：

```
new FileWriter(fWrite,true);
```

16.5 扩展练习

改进程序，使得能统计出每个学生的平均成绩。

实验17 多线程处理

17.1 实验目的

当 Java 程序包含图形用户界面（GUI）时，Java 虚拟机在运行应用程序时会自动启动更多的线程，其中有两个重要的线程：AWT-EventQueue 和 AWT-Window。AWT-EventQueue 线程负责处理 GUI 事件。AWT-Window 线程负责将窗体或组件绘制到桌面。JVM 要保证各个线程都有使用 CPU 资源的机会，例如，程序中发生 GUI 界面事件时，JVM 就会将 CPU 资源切换给 AWT-EventQueue 线程，AWT-EventQueue 线程就会来处理这个事件，例如，你单击了程序中的按钮，触发 ActionEvent 事件，AWT-EventQueue 线程就立刻排队等候执行处理事件的代码。本试验的目的是掌握在 GUI 中使用 Thread 的子类创建线程。

17.2 实验要求

编写一个 Java 应用程序，在主线程中再创建一个 Frame 类型的窗口，在该窗口中再创建一个线程 giveWord。线程 giveWord 每隔 2 秒钟给出一个汉字，用户使用一种汉字输入法将该汉字输入到文本框中。程序运行效果如图所示。



17.3 程序模板

请按模板要求，将【代码】替换为 Java 程序。

ThreadWordMainClass.java

```
public class ThreadWordMainClass {  
    public static void main(String args[]) {  
        new ThreadFrame().setTitle("汉字打字练习");  
    }  
}
```

WordThread.java

```
import javax.swing.JTextField;  
public class WordThread extends Thread {  
    char word;  
    int startPosition = 19968; //Unicode表的 19968位置至 32320上的汉字
```

```

int endPosition = 32320;
JTextField showWord;
int sleepLength = 6000;
public void setJTextField(JTextField t) {
    showWord = t;
    showWord.setEditable(false);
}
public void setSleepLength(int n){
    sleepLength = n;
}
public void run() {
    int k=startPosition;
    while(true) {
        word=(char)k;
        showWord.setText(""+word);
        try{
            【代码 1】//调用 sleep方法使得线程中断 sleepLength毫秒
        }
        catch(InterruptedException e){}
        k++;
        if(k>=endPosition)
            k=startPosition;
    }
}
}

```

ThreadFrame.java

```

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class ThreadFrame extends JFrame implements ActionListener {
    JTextField showWord;
    JButton button;
    JTextField inputText,showScore;
    【代码 2】//用 WordThread声明一个 giveWord线程象
    int score = 0;
    ThreadFrame() {
        showWord = new JTextField(6);
        showWord.setFont(new Font("",Font.BOLD,72));
        showWord.setHorizontalAlignment(JTextField.CENTER );
        【代码 3】//创建 giveWord线程
        giveWord.setJTextField(showWord);
        giveWord.setSleepLength(5000);
        button = new JButton("开始");
        inputText = new JTextField(10);
        showScore = new JTextField(5);
        showScore.setEditable(false);
        button.addActionListener(this);
        inputText.addActionListener(this);
        add(button,BorderLayout.NORTH);
        add(showWord,BorderLayout.CENTER);
        JPanel southP=new JPanel();
        southP.add(new JLabel("输入汉字（回车）："));
        southP.add(inputText);
        southP.add(showScore);
        add(southP,BorderLayout.SOUTH);
        setBounds(100,100,350,180);
        setVisible(true);
        validate();
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```

```

public void actionPerformed(ActionEvent e) {
    if(e.getSource()==button) {
        if(!(giveWord.isAlive())){
            【代码 4】//创建 giveWord
            giveWord.setJTextField(showWord);
            giveWord.setSleepLength(5000);
        }
        try {
            【代码 5】//giveWord调用方法 start()
        }
        catch(Exception exe){}
    }
    else if(e.getSource()==inputText) {
        if(inputText.getText().equals(showWord.getText()))
            score++;
        showScore.setText("得分:"+score);
        inputText.setText(null);
    }
}
}
}

```

17.4 实验指导

AWT-Windows 线程负责将窗体或组件绘制到桌面。发生 GUI 界面事件时，JVM 就会将 CPU 资源切换给 AWT-EventQueue 线程。

17.5 扩展练习

17.5.1

编写一个英文单词打字游戏。要求实现编辑一个英文单词组成的文本文件（单词用空格分隔），Widthread 线程使用 Scanner 类的实例解析文本文件中的单词。

17.5.2

输入以下 Java 应用程序，运行程序，分析程序的运行结果。

TwoThreadsTest.java

```

class SimpleThread extends Thread {
    public SimpleThread(String str) {
        super(str); //调用其父类的构造方法
    }
    public void run() { //重写 run方法
        for (int i = 0; i < 10; i++) {
            System.out.println(i + " " + getName());
            //打印次数和线程的名字
            try {
                sleep((int)(Math.random() * 1000));
                //线程睡眠，把控制权交出去
            }
            catch (InterruptedException e) { }
        }
        System.out.println("DONE! " + getName());
        //线程执行结束
    }
}

public class TwoThreadsTest {
    public static void main (String args[]) {

```



```
new SimpleThread("First").start();  
//第一个线程的名字为 First  
new SimpleThread("Second").start();  
//第二个线程的名字为 Second  
}  
}
```

实验18 实验报告（二） 题目

18.1 实验目的

- 熟悉 Java 图形界面的基本设计。
- 熟悉 Java 界面的菜单使用方法。
- 熟悉 Java 的多线程应用程序开发方法。

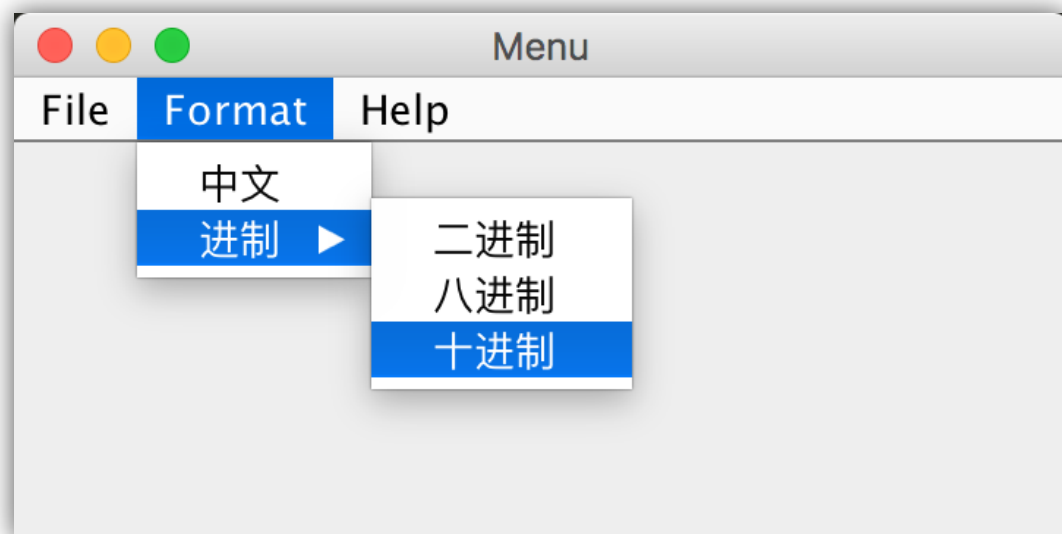
18.2 实验任务 1

编写 Java 应用程序，实现以下登陆界面（需注意密码框输入的内容不显示明文）：



18.3 实验任务 2

编写 Java 应用程序，实现以下界面：



18.4 实验任务 3

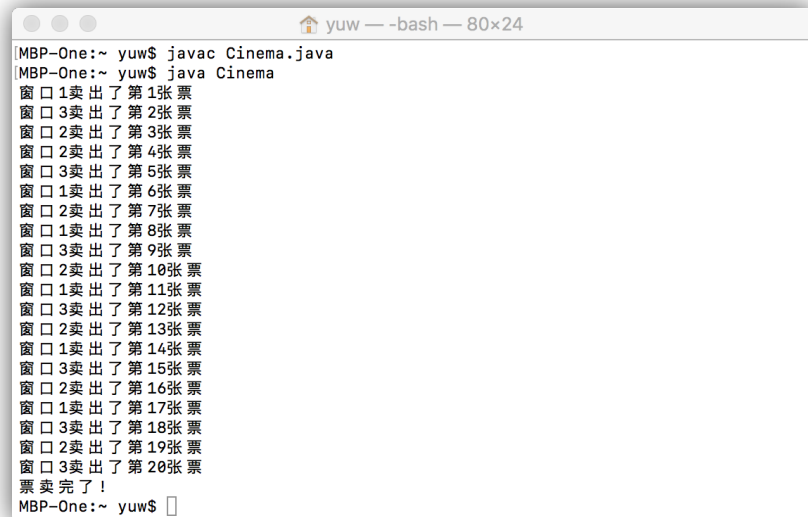
编写一个 Java 多线程应用程序，完成三个售票窗口同时出售 20 张票。具体要求如下：

- 票数要使用同一个静态值；
- 为保证不会出现卖出同一个票数，要 java 多线程同步锁。

设计思路：

- 创建一个站台类 Station，继承 Thread，重写 run 方法，在 run 方法里面执行售票操作。售票要使用同步锁：即有一个站台卖这张票时，其他站台要等这张票卖完。
- 创建主方法调用类。

运行效果参考下图：



```
MBP-One:~ yuw$ javac Cinema.java
MBP-One:~ yuw$ java Cinema
窗口 1 卖出了 第 1 张 票
窗口 3 卖出了 第 2 张 票
窗口 2 卖出了 第 3 张 票
窗口 2 卖出了 第 4 张 票
窗口 3 卖出了 第 5 张 票
窗口 1 卖出了 第 6 张 票
窗口 2 卖出了 第 7 张 票
窗口 1 卖出了 第 8 张 票
窗口 3 卖出了 第 9 张 票
窗口 2 卖出了 第 10 张 票
窗口 1 卖出了 第 11 张 票
窗口 3 卖出了 第 12 张 票
窗口 2 卖出了 第 13 张 票
窗口 1 卖出了 第 14 张 票
窗口 3 卖出了 第 15 张 票
窗口 2 卖出了 第 16 张 票
窗口 1 卖出了 第 17 张 票
窗口 3 卖出了 第 18 张 票
窗口 2 卖出了 第 19 张 票
窗口 3 卖出了 第 20 张 票
票 卖 完 了 !
MBP-One:~ yuw$
```

第三部分

JAVA 综合应用实验

实验19 网络应用

19.1 实验目的

通过实践体会网络套接字是基于 TCP 协议的有连接通信，套接字连接就是客户端的套接字对象和服务端端的套接字对象通过输入输出流连接在一起。熟悉怎样在服务器上建立 ServerSocket 对象，并让 serversocket 对象负责等待客户端请求建立套接字连接。熟悉客户端怎样建立 Socket 对象、向服务器发出套接字连接请求。

19.2 实验要求

客户端和服务端建立套接字连接后，客户将如下格式的账单发送给服务器：

"房租:2189元 水费:112.9元 电费:569元 物业费:832元"

服务器返回给客户的信息是：

您的账单："房租:2189元 水费:112.9元 电费:569元 物业费:832元
总额:3702.9元"

```
[MBP-One:exp yuw$ java ServerItem
正在等待客户
客户的地址:/127.0.0.1
正在监听
正在等待客户
2189.0
112.9
569.0
832.0
```

```
[MBP-One:exp yuw$ java ClientItem
输入服务器的IP:127.0.0.1
输入端口号:4331
输入账单：
房租:2189元      水费:112.9元      电费:569元      物业费:832元
您的账单：
账单房租:2189元      水费:112.9元      电费:569元      物业费:832元
总额:3702.9元
```

19.3 程序模板

请按模板要求，将【代码】替换为 Java 程序。

ClientItem.java

```
import java.io.*;
import java.net.*;
import java.util.*;
public class ClientItem {
    public static void main(String args[]) {
        Scanner scanner = new Scanner(System.in);
        Socket clientSocket = null;
        DataInputStream inData = null;
        DataOutputStream outData = null;
        Thread thread;
        Read read = null;
        try{
            clientSocket = new Socket();
            read = new Read();
            thread = new Thread(read);    //负责读取信息的线程
            System.out.print("输入服务器的 IP:");
            String IP = scanner.nextLine();
            System.out.print("输入端口号:");
            int port = scanner.nextInt();
            String enter = scanner.nextLine(); //消耗回车
            if(clientSocket.isConnected()){
            }
            else{
                InetAddress address = InetAddress.getByName(IP);
                InetSocketAddress socketAddress=new InetSocketAddress(address,port);
                clientSocket.connect(socketAddress);
                InputStream in = 【代码 1】//clientSocket调用 getInputStream()返回输入流
                OutputStream out = 【代码 2】//clientSocket调用 getOutputStream()返回输出流
                inData = new DataInputStream(in);
                outData = new DataOutputStream(out);
                read.setDataInputStream(inData);
                read.setDataOutputStream(outData);
                thread.start();    //启动负责读信息的线程
            }
        }
        catch(Exception e) {
            System.out.println("服务器已断开"+e);
        }
    }
}

class Read implements Runnable {
    Scanner scanner = new Scanner(System.in);
    DataInputStream in;
    DataOutputStream out;
    public void setDataInputStream(DataInputStream in) {
        this.in = in;
    }
    public void setDataOutputStream(DataOutputStream out) {
        this.out = out;
    }
    public void run() {
        System.out.println("输入账单:");
        String content = scanner.nextLine();
        try{
            out.writeUTF("账单"+content);
            String str = in.readUTF();
            System.out.println(str);
            str = in.readUTF();
            System.out.println(str);
            str = in.readUTF();
            System.out.println(str);
        }
        catch(Exception e) {}
    }
}
```

ServerItem.java

```
import java.io.*;
import java.net.*;
import java.util.*;
public class ServerItem {
    public static void main(String args[]) {
        ServerSocket server = null;
        ServerThread thread;
        Socket you = null;
        while(true) {
            try{
                server = 【代码 3】 //创建在端口 4331上负责监听的 ServerSocket对象
            }
            catch(IOException e1) {
                System.out.println("正在监听");
            }
            try{
                System.out.println("正在等待客户");
                you = 【代码 4】 // server调用 accept()返回和客户端相连接的 Socket对象
                System.out.println("客户的地址:"+you.getInetAddress());
            }
            catch (IOException e) {
                System.out.println(""+e);
            }
            if(you!=null) {
                new ServerThread(you).start();
            }
        }
    }
}

class ServerThread extends Thread {
    Socket socket;
    DataInputStream in = null;
    DataOutputStream out = null;
    ServerThread(Socket t) {
        socket = t;
        try {
            out = new DataOutputStream(socket.getOutputStream());
            in = new DataInputStream(socket.getInputStream());
        }
        catch(IOException e) {}
    }
    public void run() {
        try{
            String item = in.readUTF();
            Scanner scanner = new Scanner(item);
            scanner.useDelimiter("[^0123456789.]+");
            if(item.startsWith("账单")) {
                double sum = 0;
                while(scanner.hasNext()){
                    try{
                        double price = scanner.nextDouble();
                        sum = sum + price;
                        System.out.println(price);
                    }
                    catch(InputMismatchException exp){
                        String t = scanner.next();
                    }
                }
                out.writeUTF("您的账单:");
                out.writeUTF(item);
                out.writeUTF("总额:"+sum+"元");
            }
        }
        catch(Exception exp){}
    }
}
```

```
}
```

19.4 实验指导

可以使用 `Socket` 类不带参数的构造方法 `public socket()` 创建一个套接字对象，该对象不请求任何连接。该对象再调用 `public void connect(SocketAddress endpoint) throws IOException` 请求和参数 `SocketAddress` 指定地址的套接字建立连接。为了使用 `connect` 方法，可以使用 `SocketAddress` 的子类 `InetSocketAddress` 创建一个对象，`InetSocketAddress` 的构造方法是：
`public InetSocketAddress(InetAddress addr, int port)。`

19.5 扩展练习

19.5.1 改进服务器程序计算体积

改进服务器端程序，使得用户还可以发送如下格式的货品明细给服务器：

货品 宽 90厘米 高 69厘米 长 156厘米

服务器返回给客户的信息是：

货品 宽 90厘米 高 69厘米 长 156厘米

体积：968760立方厘米

19.5.2 解析 URL

输入下面的 Java 应用程序，运行程序，分析运行结果。

ParseURL.java

```
import java.net.*;
import java.io.*;
public class ParseURL {
    public static void main(String[] args) throws Exception {
        URL aURL = new URL("http://localhost:200/text.txt#BOTTOM");
        System.out.println(aURL);
        System.out.println("protocol = " + aURL.getProtocol());
        System.out.println("host = " + aURL.getHost());
        System.out.println("filename = " + aURL.getFile());
        System.out.println("port = " + aURL.getPort());
        System.out.println("ref = " + aURL.getRef());
    }
}
```

实验20 实验报告（三） 题目

20.1 实验目的

熟悉 Java 综合应用程序的开发。

20.2 实验任务

编写一个 Java 应用程序，实现图形界面多人聊天室（多线程实现），要求聊天室窗口标题是“欢迎使用 XXX 聊天室应用”，其中 XXX 是自己的班级姓名学号，如“软件 171 张三 1234”。